# REGAETuning out of Rabbit Holes in Mixed-Motive, Contextual Bandit Problems (Technical Appendix)

Ethan Pedersen
Brigham Young University
Provo, UT, United States
ethanp55@byu.edu

Jacob Crandall
Brigham Young University
Provo, UT, United States
crandall@cs.byu.edu

**KEYWORDS**

Offline Learning, Contextual Bandits, Mixed Motives

## 1 CODE

We provide our code in the supplementary "code" folder. There is a readme.txt file that describes the code layout. Our language of choice was Python 3.10. Neural networks were implemented with TensorFlow 2.16 [1]. Other machine learning algorithms, such as K-Nearest Neighbors (for AlegAATr), were implemented with scikit-learn 1.4 [7].

## 2 NETWORK ARCHITECTURES

This section contains figures for the neural network architectures we used.

### 2.1 Algorithms With One Set of Inputs

Algorithms with one set of inputs (raw state features or AAT features [4]) use the same network architecture (for comparison purposes). Specifically, EG-Raw, EG-AAT, REGAE-Raw, and REGAE-AAT use the architecture shown in Figure 1.

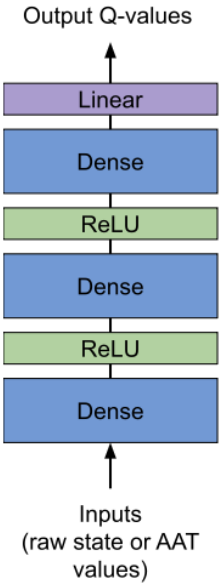



**Figure 1: Architecture for one set of inputs.**

### 2.2 Algorithms With RawAAT Features

We remind the reader that RawAAT features include both raw and AAT values. Figure 2 shows the network architecture of algorithms that use RawAAT features (EG-RawAAT and REGAE-RawAAT).

### 2.3 MADQN

We experimented with a version of a multi-agent deep Q-network (MADQN) [3]. MADQN is a traditional learner; due to results similar to those of other traditional learners, we did not present MADQN's results in the main paper. Its architecture is nearly the same as EG-Raw/DQN (Figure 1). However, there are two main differences: (1) the output dimension is 1 (i.e., it predicts a single value) and (2) MADQN uses a distinct neural network for each one of its generators (i.e., it uses $g$ networks, where $g$ is the number of generators).

### 2.4 RDQN

RDQN, a "really deep Q-network", is another traditional learner that is also similar to EG-Raw/DQN but simply uses more layers. We felt it would be worthwhile to explore whether a deeper architecture would affect results. We found that, similar to MADQN, results were very similar to those of the other traditional learners, which is why we also did not present RDQN in the main paper. Its architecture is shown in Figure 3.

### 2.5 PPO

Instead of Q-learning, Proximal Policy Optimization (PPO) [8] aims to directly optimize the agent's policy; it does so with the use of an action model (the main model wished to be learned) and a value model. We label PPO as a traditional learner, though the training process is slightly different than $\epsilon$-greedy. Because PPO models and samples from probabilities over actions (in our case, generators), exploration via an $\epsilon$ value is not needed. Aside from this fact, training PPO is identical to the traditional approach described in the main paper. Our motivation for using PPO was to explore a popular and somewhat new algorithm and analyze whether PPO's loss clipping and differences in training would impact the learning outcomes of traditional learning in mixed-motive domains. We found that results for PPO were similar to other traditional learners, one of the reasons why we did not discuss it further in the main paper. Additionally, due to PPO's slower training times, we found it

*Proc.@ of the Adaptive and Learning Agents Workshop (ALA 2026), Aydeniz, Delgrange, Mohammedalamen, Yang (eds.), May 25 – 26, 2026, Paphos, Cyprus, https://alaworkshop2026.github.io/.* 2026.

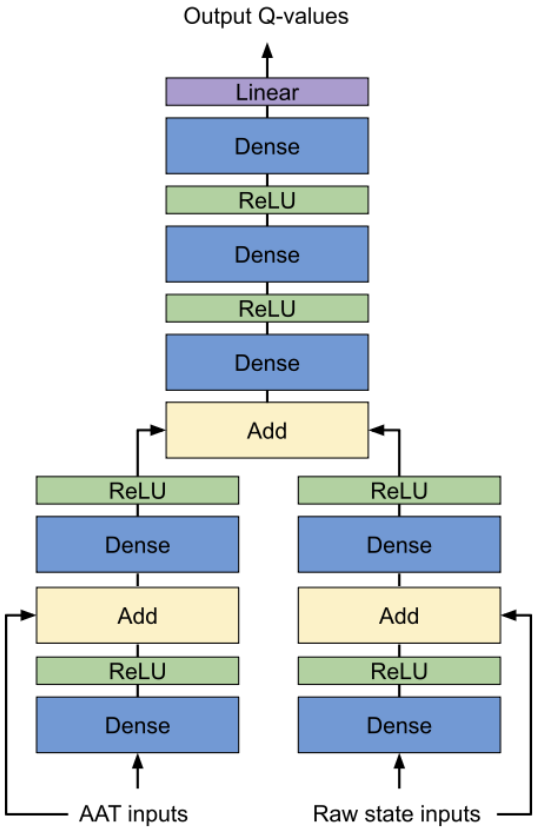



**Figure 2: Architecture for RawAAT features.**

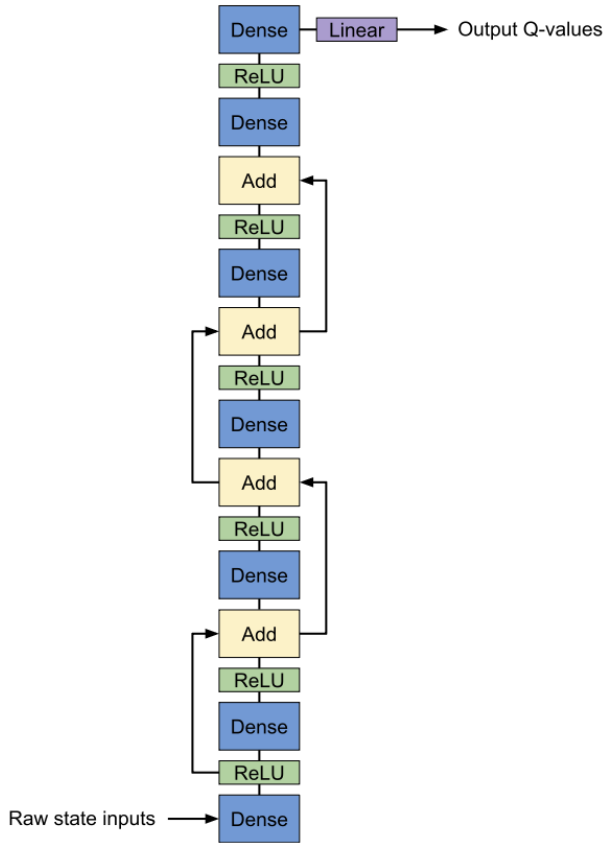



**Figure 3: Architecture for RDQN, a deeper version of DQN.**

infeasible to train/learn in the complex Junior High Game. Figure 4 shows the architectures we used for PPO's action and value models, respectively.

## 3 ADAPTABILITY SCORES

Here we include adaptability scores for each algorithm, including RDQN, MADQN, and PPO (three algorithms we did not present in the main paper); note that RDQN and MADQN were trained with EG and PPO was not trained with REGAETune. Please also note that we did not obtain results for PPO in the JHG, as training times were too long; due to these long training times, we also only generated results for one epoch of re-training and re-testing (whereas the table in the main paper shows average results for 30 epochs). The values are presented in Table 1. While these are only for a single epoch, the trend is similar to that found in the main paper (REGAETune algorithms tend to be more adaptable, especially when compared to their EG counterparts).

## 4 PRISONER'S DILEMMA

This section provides additional details and results for the repeated prisoner's dilemma. Note that previous AAT work has been done in this domain; accordingly, we borrowed some of the generators, assumptions, and checkers from [6].

### 4.1 Generators

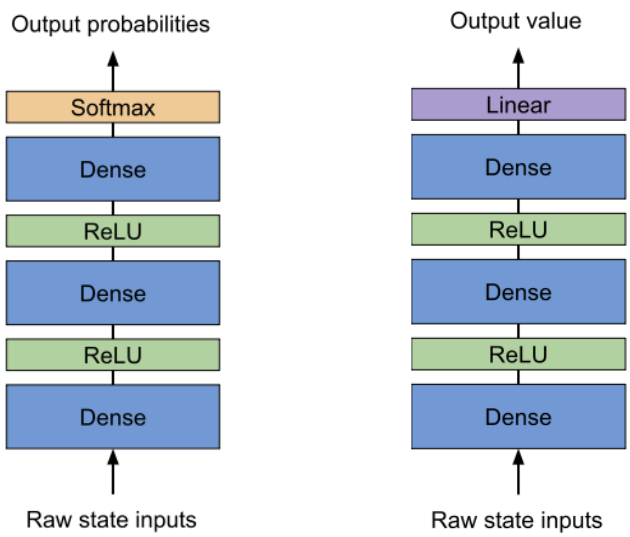
Generator descriptions are given in the main paper and are the same as those used in [6]. See that cited paper or our code for implementation details.

### 4.2 AAT Assumptions and Checkers

The assumptions and alignment checkers we used are somewhat similar to those found in [6], with a few differences. Specific details for each generator are given in Table 2. Note that the table spans multiple pages, as we wanted to be as thorough as possible.

**Table 1: Competitive scores ($\mathcal{S}_{\text{comp}}$), cooperative scores ($\mathcal{S}_{\text{coop}}$), and adaptability score ($\mathcal{S}_A$) for each algorithm in each domain. Note that, because of long training times required for PPO, these results are for just a single trained model (one epoch). As such, the values differ from those shown in Table 3 of the main paper. As these results come from less evaluations, they are intended to be illustrative of the behavior of different neural architectures, rather than a rigorous comparison.**

| | Prisoner's Dilemma | | | Grid Stag Hung | | | JHG | | |
|---|---|---|---|---|---|---|---|---|---|
| | $\mathcal{S}_{\text{comp}}$ | $\mathcal{S}_{\text{coop}}$ | $\mathcal{S}_A$ | $\mathcal{S}_{\text{comp}}$ | $\mathcal{S}_{\text{coop}}$ | $\mathcal{S}_A$ | $\mathcal{S}_{\text{comp}}$ | $\mathcal{S}_{\text{coop}}$ | $\mathcal{S}_A$ |
| **EG-Raw** | 1.000 | 0.224 | 0.224 | 1.000 | 0.169 | 0.169 | 0.779 | 0.003 | 0.003 |
| **EG-AAT** | 1.000 | 0.210 | 0.210 | 1.000 | 0.249 | 0.249 | 1.000 | 0.308 | 0.308 |
| **EG-RawAAT** | 1.000 | 0.300 | 0.300 | 0.691 | 0.297 | 0.297 | 0.560 | 0.645 | 0.560 |
| **MADQN** | 1.000 | 0.338 | 0.338 | 1.000 | 0.341 | 0.341 | 0.781 | 0.001 | 0.001 |
| **RDQN** | 1.000 | 0.286 | 0.286 | 1.000 | 0.281 | 0.281 | 0.770 | 0.259 | 0.259 |
| **PPO** | 0.881 | 0.112 | 0.112 | 1.000 | 0.185 | 0.185 | N.A. | N.A. | N.A. |
| **REGAE-Raw** | 0.500 | 0.939 | 0.500 | 0.987 | 0.349 | 0.349 | 0.677 | 0.558 | 0.558 |
| **REGAE-AAT** | 0.833 | 0.756 | 0.756 | 0.395 | 0.525 | 0.395 | 1.000 | 0.492 | 0.492 |
| **REGAE-RawAAT** | 0.600 | 0.472 | 0.472 | 0.395 | 0.477 | 0.395 | 0.700 | 0.580 | 0.580 |
| **AlegAATr** | 0.933 | 0.810 | 0.810 | 0.642 | 0.413 | 0.413 | 0.846 | 0.520 | 0.520 |




**(a) Action** **(b) Value**

**Figure 4: Architectures for PPO's action and value models, respectively.**

**Table 2: Prisoner's dilemma assumptions and checkers.**

| Generator | Assumptions | Checkers |
|---|---|---|
| Bullied | The associate insists that agent's average reward is less than or equal to what it would receive in the bullied payoff. | $x_t^{\text{bd_insists}} = \begin{cases} 0.8x_{t-1}^{\text{bd_insists}} + 0.2 & \text{if } \bar{R}_t \leq \bar{V}_{\text{bullied}} \\ 0.8x_{t-1}^{\text{bd_insists}} & \text{otherwise} \end{cases}$ where $\bar{V}_{\text{bullied}}$ is the average per-round payoff the agent receives in this solution and $\bar{R}_t$ is the average per-round payoff the agent has received up to time $t$. |
| | The associate will not harm/punish the agent if the agent conforms with the bullied target solution. | $x_t^{\text{bd_notHarm}} = \begin{cases} 0.8x_{t-1}^{\text{bd_notHarm}} + 0.2 & \text{if } r_t \geq V_{\text{bd}} \\ 0.7x_{t-1}^{\text{bd_notHarm}} & \text{otherwise} \end{cases}$ where $V_{\text{bd}}$ is the payoff the agent should get in the round (in the target solution). If Bullied is not followed in round $t$, then $x_t^{\text{bd_notHarm}} = x_{t-1}^{\text{bd_notHarm}}$. |
| | The agent's average rewards when using the Bullied generator are greater than or equal to the agent's average overall rewards. | $x_t^{\text{bd_usage}} = \begin{cases} 1 - \frac{\bar{R}_t}{\bar{R}_t^{bd}} & \text{if } \bar{R}_t^{bd} \geq \bar{R}_t \\ 0.0 & \text{otherwise} \end{cases}$ where $\bar{R}_t$ is the agent's average per-round reward up to time $t$ and $\bar{R}_t^{bd}$ is the agent's average per-round reward up to time $t$ but only for rounds where it has used the Bullied generator. |
| | The agent's average overall rewards are greater than or equal to the agent's average rewards when using the Bullied generator. | $x_t^{\text{bd_noUsage}} = \begin{cases} 1 - \frac{\bar{R}_t^{bd}}{\bar{R}_t} & \text{if } \bar{R}_t \geq \bar{R}_t^{bd} \\ 0.0 & \text{otherwise} \end{cases}$ |
| Bully-Punish | The associate is willing to be bullied. | $x_t^{\text{B_willing1}} = \begin{cases} 0.7x_{t-1}^{\text{B_willing1}} + 0.3 & \text{if cooperated} \\ 0.3x_{t-1}^{\text{B_willing1}} & \text{otherwise} \end{cases}$ where *cooperated* indicates that the associate played its portion of the target solution. |
| | In rounds where Bully-Punish was used, the associate was willing to be bullied. | $x_t^{\text{B_willing2}} = \begin{cases} 0.7x_{t-1}^{\text{B_willing2}} + 0.3 & \text{if cooperated} \\ 0.7x_{t-1}^{\text{B_willing2}} & \text{otherwise} \end{cases}$ When Bully-Punish is not used, then $x_t^{\text{B_willing2}} = x_{t-1}^{\text{B_willing2}}$. |
| | Punishment convinces the associate to be bullied. | $x_t^{\text{B_responds}} = \begin{cases} 0.5x_{t-1}^{\text{B_responds}} + 0.5 & \text{if cooperated} \\ 0.5x_{t-1}^{\text{B_responds}} & \text{otherwise} \end{cases}$ If the agent did not punish in the current or previous round, then $x_t^{\text{B_responds}} = x_{t-1}^{\text{B_responds}}$. |
| | The agent's average rewards when using the Bully-Punish generator are greater than or equal to the agent's average overall rewards. | $x_t^{\text{B_usage}} = \begin{cases} 1 - \frac{\bar{R}_t}{\bar{R}_t^{B}} & \text{if } \bar{R}_t^{B} \geq \bar{R}_t \\ 0.0 & \text{otherwise} \end{cases}$ where $\bar{R}_t$ is the agent's average per-round reward up to time $t$ and $\bar{R}_t^{B}$ is the agent's average per-round reward up to time $t$ but only for rounds where it has used the Bully-Punish generator. |
| | The agent's average overall rewards are greater than or equal to the agent's average rewards when using the Bully-Punish generator. | $x_t^{\text{B_noUsage}} = \begin{cases} 1 - \frac{\bar{R}_t^{B}}{\bar{R}_t} & \text{if } \bar{R}_t \geq \bar{R}_t^{B} \\ 0.0 & \text{otherwise} \end{cases}$ |

| | | |
|---|---|---|
| CFR | The associate played a myopic best response. | $x_t^{\text{cfr_br1}} = 0.8x_{t-1}^{\text{cfr_br1}} + 0.2k$,<br>where $k = 1$ if the associate played a bast response to the agent's strategy in that round and $k = 0$ otherwise. |
| | The associate played a myopic best response in rounds where the agent used the CFR generator. | $x_t^{\text{cfr_br2}} = \begin{cases} 0.8x_{t-1}^{\text{cfr_br2}} + 0.2k & \text{if agent used CFR} \\ x_{t-1}^{\text{cfr_br2}} & \text{otherwise} \end{cases}$ |
| | The agent's average rewards when using the CFR generator are greater than or equal to the agent's average overall rewards. | $x_t^{\text{cfr_usage}} = \begin{cases} 1 - \frac{\bar{R}_t}{\bar{R}_t^{cfr}} & \text{if } \bar{R}_t^{cfr} \geq \bar{R}_t \\ 0.0 & \text{otherwise} \end{cases}$<br>where $\bar{R}_t$ is the agent's average per-round reward up to time $t$ and $\bar{R}_t^{cfr}$ is the agent's average per-round reward up to time $t$ but only for rounds where it has used the CFR generator. |
| | The agent's average overall rewards are greater than or equal to the agent's average rewards when using the CFR generator. | $x_t^{\text{cfr_noUsage}} = \begin{cases} 1 - \frac{\bar{R}_t^{cfr}}{\bar{R}_t} & \text{if } \bar{R}_t \geq \bar{R}_t^{cfr} \\ 0.0 & \text{otherwise} \end{cases}$ |
| Coop | The associate wants to cooperate. | $x_t^{\text{c_wants}} = \begin{cases} 0.2x_{t-1}^{\text{c_wants}} + 0.8 & \text{if nice} \\ 0.9x_{t-1}^{\text{c_wants}} + 0.1 & \text{if cooperated} \\ 0.8x_{t-1}^{\text{c_wants}} & \text{otherwise} \end{cases}$<br>where *nice* indicates that the associate played in a way that would have boosted the agent's payoffs while not benefiting themselves (did worse than the value they would have gotten from mutual cooperation), and *cooperated* indicates the associate played the cooperative action. |
| | The associate cooperates when the agent cooperates. | $x_t^{\text{c_mirrors}} = \begin{cases} 0.7x_{t-1}^{\text{c_mirrors}} + 0.3 & \text{if cooperated} \\ 0.7x_{t-1}^{\text{c_mirrors}} & \text{otherwise} \end{cases}$<br>When Coop is not used in a round, then $x_t^{\text{c_mirrors}} = x_{t-1}^{\text{c_mirrors}}$. |
| | The associate exploits the agent when the agent cooperates. | $x_t^{\text{c_notexploit}} = \begin{cases} 0.7x_{t-1}^{\text{c_notexploit}} & \text{if } r_t^{-i} > \bar{V}^{-i} \\ 0.8x_{t-1}^{\text{c_notexploit}} + 0.2 & \text{otherwise} \end{cases}$<br>where $r_t^{-i}$ is the associates payoff in round $t$ and $\bar{V}^{-i}$ is the payoff the associate should get when it cooperates in the round. When Coop is not used in a round, then $x_t^{\text{c_notexploit}} = x_{t-1}^{\text{c_notexploit}}$. |
| | The agent's average rewards when using the Coop generator are greater than or equal to the agent's average overall rewards. | $x_t^{\text{c_usage}} = \begin{cases} 1 - \frac{\bar{R}_t}{\bar{R}_t^c} & \text{if } \bar{R}_t^c \geq \bar{R}_t \\ 0.0 & \text{otherwise} \end{cases}$<br>where $\bar{R}_t$ is the agent's average per-round reward up to time $t$ and $\bar{R}_t^c$ is the agent's average per-round reward up to time $t$ but only for rounds where it has used the Coop generator. |
| | The agent's average overall rewards are greater than or equal to the agent's average rewards when using the Coop generator. | $x_t^{\text{c_noUsage}} = \begin{cases} 1 - \frac{\bar{R}_t^c}{\bar{R}_t} & \text{if } \bar{R}_t \geq \bar{R}_t^c \\ 0.0 & \text{otherwise} \end{cases}$ |

| | | |
|---|---|---|
| Coop-Punish | The associate cooperates when the agent uses the Coop-Punish generator. | $x_t^{\text{cp_mirrors}} = \begin{cases} 0.7x_{t-1}^{\text{cp_mirrors}} + 0.3 & \text{if cooperated} \\ 0.7x_{t-1}^{\text{cp_mirrors}} & \text{otherwise} \end{cases}$ When Coop-Punish is not used in a round, then $x_t^{\text{cp_mirrors}} = x_{t-1}^{\text{cp_mirrors}}$. |
| | Punishment convinces the associate to cooperate. | $x_t^{\text{cp_responds}} = \begin{cases} 0.5x_{t-1}^{\text{c_responds}} + 0.5 & \text{if cooperated} \\ 0.5x_{t-1}^{\text{c_responds}} & \text{otherwise} \end{cases}$ If the agent did not punish in the current or previous round, then $x_t^{\text{c_responds}} = x_{t-1}^{\text{c_responds}}$. |
| | The agent's average rewards when using the Coop-Punish generator are greater than or equal to the agent's average overall rewards. | $x_t^{\text{cp_usage}} = \begin{cases} 1 - \frac{\bar{R}_t}{\bar{R}_t^{cp}} & \text{if } \bar{R}_t^{cp} \geq \bar{R}_t \\ 0.0 & \text{otherwise} \end{cases}$ where $\bar{R}_t$ is the agent's average per-round reward up to time $t$ and $\bar{R}_t^{cp}$ is the agent's average per-round reward up to time $t$ but only for rounds where it has used the Coop-Punish generator. |
| | The agent's average overall rewards are greater than or equal to the agent's average rewards when using the Coop-Punish generator. | $x_t^{\text{cp_noUsage}} = \begin{cases} 1 - \frac{\bar{R}_t^{cp}}{\bar{R}_t} & \text{if } \bar{R}_t \geq \bar{R}_t^{cp} \\ 0.0 & \text{otherwise} \end{cases}$ |
| Minimax | The associate plays the attack strategy. | $x_t^{\text{mx_harm}} = \begin{cases} 0.8x_{t-1}^{\text{mx_harm}} + 0.2 & \text{if } r_t < r^{\text{mm}} \\ 0.8x_{t-1}^{\text{mx_harm}} & \text{otherwise} \end{cases}$ where $r_t$ is the round payoff obtained by the agent in round $t$ and $r^{\text{mm}}$ is its minimax value. |
| | The agent's average rewards when using the Minimax generator are greater than or equal to the agent's average overall rewards. | $x_t^{\text{mx_usage}} = \begin{cases} 1 - \frac{\bar{R}_t}{\bar{R}_t^{mx}} & \text{if } \bar{R}_t^{mx} \geq \bar{R}_t \\ 0.0 & \text{otherwise} \end{cases}$ where $\bar{R}_t$ is the agent's average per-round reward up to time $t$ and $\bar{R}_t^{mx}$ is the agent's average per-round reward up to time $t$ but only for rounds where it has used the Minimax generator. |
| | The agent's average overall rewards are greater than or equal to the agent's average rewards when using the Minimax generator. | $x_t^{\text{mx_noUsage}} = \begin{cases} 1 - \frac{\bar{R}_t^{mx}}{\bar{R}_t} & \text{if } \bar{R}_t \geq \bar{R}_t^{mx} \\ 0.0 & \text{otherwise} \end{cases}$ |

## 4.3 Raw Features

The state of the prisoner's dilemma does not contain much information due to the game's simplistic nature. Accordingly, the raw feature vectors we used were simply defined as $\langle a_t^i, a_t^{-i}, r_t^i, r_{t-1}^i \rangle$, where $a_t^i$ is player $i$'s action index (0 for cooperate, 1 for compete) in round $t$, $a_t^{-i}$ is the other player's action index in round $t$, $r_t^i$ is player $i$'s reward in round $t$, and $r_{t-1}^i$ is player $i$'s reward in the previous round $t-1$.

## 4.4 Adaptability Score Components

In the main paper, we defined the adaptability metric we used, as well as the pieces it is composed of: $S_{\text{comp}}$, which is composed of $R_{\text{comp}}$, and $S_{\text{coop}}$, which is composed of $R_{\text{coop}}$. Furthermore, $R_{\text{comp}}$ is composed of $U_d^*$ and $U_{\min}$, and $R_{\text{coop}}$ is composed of $U_c^*$, and $v_{\text{mm}}$. Here we define the specific values of $U_d^*$, $U_{\min}$, $U_c^*$, and $v_{\text{mm}}$ we used in the repeated prisoner's dilemma.

The $U_d^*$ value we used was -1. The $U_{\min}$ value was -3, since the lowest reward an agent can receive in a single round is -3. To test for cooperation, we used two cooperative scenarios: (1) self-play and (2) tit-for-tat partners. The $U_c^*$ values for both conditions were 3, since an agent can receive 3 each round with mutual cooperation. The $v_{\text{mm}}$ value was -1 since an agent can guarantee itself a reward of -1 by always defecting. Note that, since we used 30 rounds of play in our testing, we multiplied each of these values by 30 when calculating the adaptability scores.

All of the values for this domain were found through the theoretical components of the game.

## 4.5 Additional Results

We present competitive, cooperative, and adaptability scores in this subsection; Figures 5-7 show average scores (with standard errors).

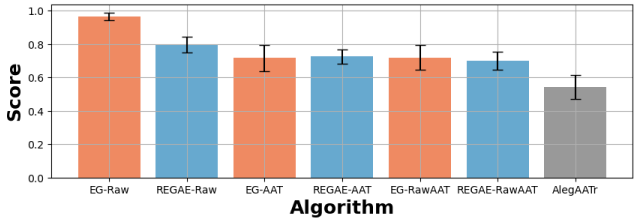



**Figure 5: Competitive scores.**

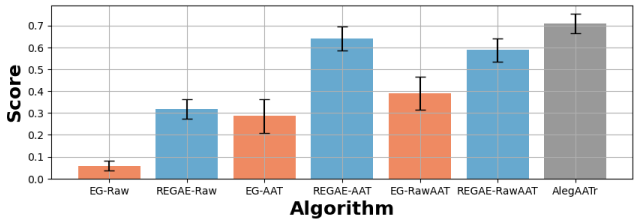



**Figure 6: Cooperative scores.**

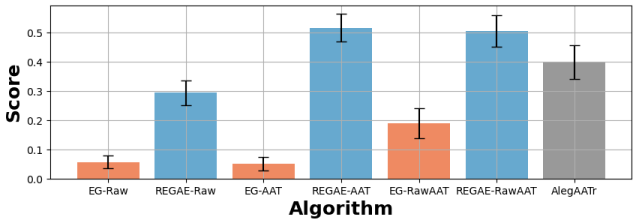



**Figure 7: Adaptability scores.**

## 4.6 Replicator Dynamic

We did not include these results in the main paper for the sake of space. One of the advantages of the prisoner's dilemma is that one can perform a replicator dynamic. A replicator dynamic can help describe how the proportion of players using certain strategies changes over time based on the success of those strategies [10]. The process we used is defined in Algorithm 1.

**Algorithm 1** Replicator dynamic in the repeated prisoner's dilemma.

Let $N$ denote the number of algorithms in the population (in our case, 10)
Let $x_i^t$ denote the proporation of the population using algorithm $i$ in generation $t$; initialize $x_i^1 = \frac{1}{N}$
Let $R(i,j) \in [0,1]$ be the expected payoff of algorithm $i$ against algorithm $j$; we used min-max scaling to yield values in this range
Let $f_i^t$ be the fitness of algorithm $i$ in generation $t$
**for** each generation $t$ **do**
  Compute the fitness of each algorithm $i \in [1,N]$: $f_i^t = \sum_{j=1}^{N} x_j^t R(i,j)$
  Compute the average fitness of the population: $\phi_t = \sum_{j=1}^{N} x_j^t f_j^t$
  Update the population share for the next generation: $x_i^{t+1} = x_i^t + x_i^t(f_i^t - \phi_t)$
**end for**

Figure 8 shows results from running 100 generations of a replicator dynamic with the seven agents from the main paper, plus MADQN, RDQN, and PPO. We can see that AlegAATr eventually dominates the population, with a proportion of 1.0; all other algorithms have proportions of 0.0.

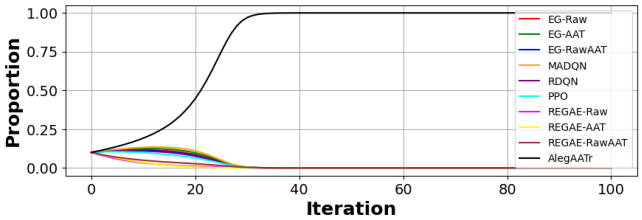



**Figure 8: AlegAATr generator vectors.**

# 5 GRID STAG HUNT

This section provides additional details and results for the Grid Stag Hunt domain.

## 5.1 Generators

We used four generators, two designed to pursue the hare and two that pursue the stag: Greedy-Hare, Greedy Planner-Hare, Greedy Planner-Stag, and Team Aware. Descriptions of these generators can be found in the main paper. Our implementations are nearly identical to those found in [2], with the exception of setting the targets of certain generators to stag or hare.

## 5.2 AAT Assumptions and Checkers

The assumptions and alignment checkers we used for each generator are described in Table 3. Please note that we used the same progress assumptions and checkers for each generator, which are described in Table 4. Note that the tables span multiple pages, as we wanted to be as thorough as possible.

**Table 3: Grid Stag Hunt assumptions and checkers.**

| Generator | Assumptions | Checkers |
|---|---|---|
| Team Aware | The agent is getting closer to the stag. | $x_t^{\text{ta_closer}} = \begin{cases} 1.0 & \text{if } d_s^t < d_s^{t-1} \\ 0.5 & \text{if } d_s^t = d_s^{t-1} \\ 0.0 & \text{otherwise} \end{cases}$ where $d_s^t$ represents the agent's shortest distance at time $t$ to greedily reach the stag. |
| | There are no collisions with other hunters. | $x_t^{\text{ta_collisions}} = 1 - \frac{c}{3}$ where $c$ represents the number of estimated collisions and 3 refers to the number of hunters. $c$ is estimated by looking at each hunter's positions at times $t$ and $t-1$. If the hunter is in the same position at both times $t$ and $t-1$ and is not neighboring a prey (stag or hare), $c$ is incremented by 1. Since there are 3 hunters, $c$ can be an integer in $[0, 3]$. |
| Greedy Planner-Stag | The agent is getting closer to the stag. | $x_t^{\text{gps_closer}} = \begin{cases} 1.0 & \text{if } d_s^t < d_s^{t-1} \\ 0.5 & \text{if } d_s^t = d_s^{t-1} \\ 0.0 & \text{otherwise} \end{cases}$ |
| | There are no collisions with other hunters. | $x_t^{\text{gps_collisions}} = 1 - \frac{c}{3}$ |
| Greedy-Hare | The agent is getting closer to the hare. | $x_t^{\text{gh_closer}} = \begin{cases} 1.0 & \text{if } d_h^t < d_h^{t-1} \\ 0.5 & \text{if } d_h^t = d_h^{t-1} \\ 0.0 & \text{otherwise} \end{cases}$ where $d_h^t$ represents the agent's shortest distance at time $t$ to greedily reach the hare. |
| Greedy Planner-Hare | The agent is getting closer to the hare. | $x_t^{\text{gph_closer}} = \begin{cases} 1.0 & \text{if } d_h^t < d_h^{t-1} \\ 0.5 & \text{if } d_h^t = d_h^{t-1} \\ 0.0 & \text{otherwise} \end{cases}$ |

**Table 4: Grid Stag Hunt progress assumptions and checkers.**

| Assumptions | Checkers |
|---|---|
| The grid width is small. | $x_t^{\text{w_small}} = \begin{cases} 1 - \frac{c}{10} & \text{if } c \leq 10 \\ 0.0 & \text{otherwise} \end{cases}$ where $c$ is the number of columns in the grid (i.e., the width). |
| The grid width is large. | $x_t^{\text{w_large}} = \begin{cases} 1 - \frac{10}{c} & \text{if } c \geq 10 \\ 0.0 & \text{otherwise} \end{cases}$ |
| The grid height is small. | $x_t^{\text{h_small}} = \begin{cases} 1 - \frac{r}{10} & \text{if } r \leq 10 \\ 0.0 & \text{otherwise} \end{cases}$ where $r$ is the number of rows in the grid (i.e., the height). |
| The grid height is large. | $x_t^{\text{h_large}} = \begin{cases} 1 - \frac{10}{r} & \text{if } r \geq 10 \\ 0.0 & \text{otherwise} \end{cases}$ |
| The agent is closer to the hare than the other hunters. | $x_t^{\text{closest_h}} = \frac{\min D_h^t}{d_h^t}$ where $d_h^t$ is the agent's shortest distance at time $t$ to greedily reach the hare and $D_h^t$ is the set containing each of the three hunter's shortest distances to greedily reach the hare. |
| The agent is closer to the stag than the other hunters. | $x_t^{\text{closest_s}} = \frac{\min D_s^t}{d_s^t}$ where $d_s^t$ is the agent's shortest distance at time $t$ to greedily reach the stag and $D_s^t$ is the set containing each of the three hunter's shortest distances to greedily reach the stag. |
| The other hunters are getting closer to the hare. | $x_t^{\text{closer_h}} = \frac{n_h^t}{2}$ where $n_h^t$ is the number of other hunters that moved closer to the hare between times $t-1$ and $t$ and 2 represents the number of other hunters. |
| The other hunters are getting closer to the stag. | $x_t^{\text{closer_s}} = \frac{n_s^t}{2}$ where $n_s^t$ is the number of other hunters that moved closer to the stag between times $t-1$ and $t$. |

## 5.3 Raw Features

We represent the raw state of the Grid Stag Hunt domain as a 14-element vector.

The first features/vector elements describe the environment/grid size. Define $r$ and $c$ as the number of rows and columns of the grid, respectively.

The next features describe the agent's current position in the grid. Define $a_1 = \frac{p_1^a}{r}$ and $a_2 = \frac{p_2^a}{c}$, where $p_1^a$ is the agent's current row position and $p_2^a$ is the agent's current column position.

Similar to the previous features, we include values to describe the current positions of the hare and stag. Define $h_1 = \frac{p_1^h}{r}$, $h_2 = \frac{p_2^h}{c}$, $s_1 = \frac{p_1^s}{r}$, and $s_2 = \frac{p_2^s}{c}$, where $p_1^h$ is the hare's current row position, $p_2^h$ is the hare's current column position, $p_1^s$ is the stag's current row position, and $p_2^s$ is the stag's current column position.

We also include features that describe the agent's distance to both the hare and stag. Let $n = r \times c$, the number of possible steps an agent can take. Let $d_a^h$ be the smallest number of steps the agent must take to greedily reach the hare. Likewise, let $d_a^s$ be the smallest number of steps the agent must take to greedily reach the stag. Define $a_h = \frac{d_a^h}{n}$ and $a_s = \frac{d_a^s}{n}$.

Finally, and similar to the previous features, we include values to describe the distances of the other hunters to both the hare and stag. Again let $n = r \times c$. Let $h_1$ and $h_2$ be the two other hunters that are not the agent. Let $d_{h_1}^h$ and $d_{h_1}^s$ be the smallest number of steps $h_1$ must take to greedily reach the hare and stag, respectively. Likewise, let $d_{h_2}^h$ and $d_{h_2}^s$ be the smallest number of steps $h_2$ must take to greedily reach the hare and stag, respectively. Define $h_{1h} = \frac{d_{h_1}^h}{n}$, $h_{1s} = \frac{d_{h_1}^s}{n}$, $h_{2h} = \frac{d_{h_2}^h}{n}$, and $h_{2s} = \frac{d_{h_2}^s}{n}$.

With these defined values in mind, our raw feature vector at any time $t$ is the 14-element vector $\langle r, c, a_1, a_2, h_1, h_2, s_1, s_2, a_h, a_s, h_{1h}, h_{1s}, h_{2h}, h_{2s} \rangle$.

## 5.4 Adaptability Score Components

Here we define the specific values of $U_d^*$, $U_{\min}$, $U_c^*$, and $v_{\text{mm}}$ we used in the Grid Stag Hunt.

The $U_d^*$ value we used was 3.377 (rounded to three digits). The $U_{\min}$ value was 0, since the lowest reward an agent can receive is 0. The $U_c^*$ values for every cooperative condition was 20, since an agent can receive 20 points each round with mutual cooperation. The $v_{\text{mm}}$ value was also 3.377 (rounded to three digits) since an agent can guarantee itself an average reward of 3.377 by always pursuing the hare.

The $U_d^*$ and $v_{\text{mm}}$ values were found by simulation. The $U_{\min}$ and $U_c^*$ values were obtained through the theoretical components of the domain.

## 5.5 Additional Results

We present competitive, cooperative, and adaptability scores in this subsection; Figures 9-11 show average scores (with standard errors).

We also show entropy scores (with standard errors) for each algorithm in Figure 12. We calculated entropy scores for each algorithm (entropy here is defined as $-\sum_{k \in \{comp, coop\}} p_k \ln(p_k)$, where $p_k$ represents the probability of competing or cooperating). Although entropy is typically minimized, particularly in classification settings [5], larger entropy values in this case are indicative of how often an algorithm cooperates or competes. The EG algorithms all produced an entropy of 0, while the REGAE algorithms produced scores near 0.3. Note that the scores for each EG algorithm are all 0 because they learned to only compete or only cooperate in each epoch.

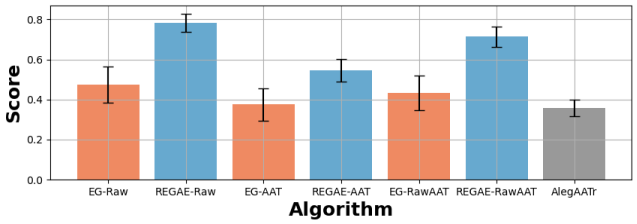



**Figure 9: Competitive scores.**

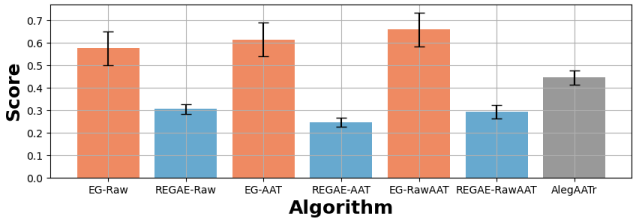



**Figure 10: Cooperative scores.**

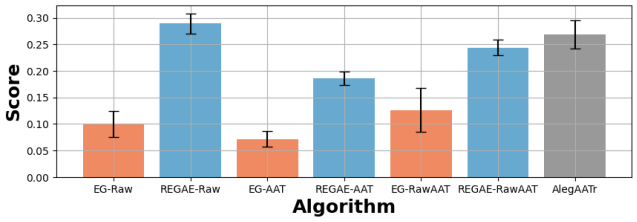



**Figure 11: Adaptability scores.**

# 6 JHG

This section provides additional details and results for the Junior High Game (JHG).

## 6.1 Generators

The set of generators each reinforcement learner could choose from while playing the JHG was composed of eight Community-Aware Behavior (CAB) agents [9], each with unique parameters to exhibit different behaviors. The first was designed to steal from players with lower popularities (i.e., the lower members in society

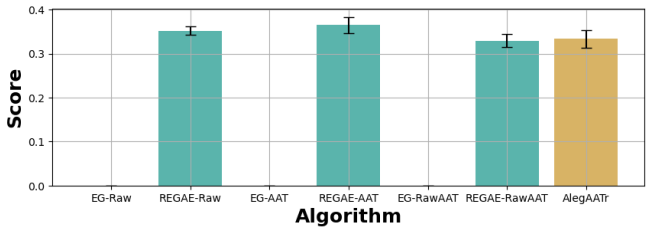



**Figure 12: Average entropy values for each algorithm in the Grid Stag Hunt. Error bars show the standard error of the mean.**

that likely cannot fight back). The second CAB was designed to play effectively against coordinated assassin agents [9] by keeping tokens for defense and then giving to others once assassin agents have low enough popularity to no longer remain a threat. The third generator was created to primarily give to "rich" members of society (i.e., those with higher popularities) while ignoring the middle and lower classes. The fourth CAB was designed to play a "normal" strategy by giving to its friends and stealing as a form of retaliation against thieves. The fifth was created to "suck up" to other players through giving, even if tokens are not reciprocated. The sixth generator always keeps its tokens, regardless of the state of the environment/society. The seventh and eighth generators use randomly selected parameters.

Specific parameter values can be found in Table 5. The CABs listed in the table follow the same order as they were described in the previous paragraph (e.g., "CAB 1" refers to the generator that steals from players with lower popularities, "CAB 2" refers to the generator that effectively plays against coordinated assassins, etc.). The meaning of the parameters can be found in the original JHG paper ([9]).

## 6.2 AAT Assumptions and Checkers

CAB agents have thirty distinct parameters they use when choosing actions (token allocations), and each parameter can have an integer value between 0 and 100. This makes for a vast parameter space ($101^{30}$ unique combinations) and, accordingly, makes understanding behavior for the numerous parameter combinations extremely difficult. Rather than attempt to observe and identify behaviors, it is arguably a more efficient and principled approach to analyze the calculations CAB agents use in their decision processes. Based on this idea, Table 6 outlines different assumption checkers: progress checkers, which are common to AAT, and five other checker types that attempt to account for the five main steps CAB agents use when determining how to allocate their tokens. The table also specifies the specific assumptions that are covered by each checker type, as well as a clarification as to whether a given assumption is CAB-specific or general across any CAB agent; if an assumption is general (i.e., not CAB-specific), an argument is given as to why we think it is still valuable. Note that the table spans multiple pages, as we wanted to be as thorough as possible.

**Table 5: CAB generator parameter values.**

| Parameter | CAB 1 | CAB 2 | CAB 3 | CAB 4 | CAB 5 | CAB 6 | CAB 7 | CAB 8 |
|---|---|---|---|---|---|---|---|---|
| Alpha | 1 | 20 | 50 | 3 | 20 | 20 | 54 | 76 |
| Debt Limits | 25 | 25 | 10 | 23 | 50 | 25 | 92 | 100 |
| Coalition Target | 0 | 70 | 55 | 77 | 40 | 70 | 72 | 63 |
| Fixed Usage | 50 | 50 | 0 | 5 | 0 | 50 | 7 | 93 |
| Modularity Weight | 100 | 100 | 80 | 12 | 70 | 100 | 66 | 48 |
| Centrality Weight | 0 | 50 | 80 | 19 | 0 | 50 | 13 | 0 |
| Collective Strength Weight | 0 | 80 | 80 | 75 | 70 | 80 | 25 | 20 |
| Familiarity Weight | 0 | 50 | 50 | 89 | 70 | 50 | 21 | 70 |
| Prosocial Weight | 0 | 70 | 0 | 63 | 25 | 70 | 5 | 97 |
| Initial Defense | 100 | 20 | 20 | 7 | 50 | 100 | 11 | 11 |
| Min Keep | 0 | 20 | 20 | 35 | 30 | 100 | 50 | 21 |
| Defense Update | 50 | 50 | 50 | 86 | 50 | 50 | 6 | 17 |
| Defense Propensity | 50 | 50 | 50 | 46 | 50 | 50 | 35 | 4 |
| Fear Defense | 0 | 0 | 50 | 39 | 25 | 0 | 75 | 97 |
| Safety First | 0 | 0 | 100 | 28 | 0 | 0 | 75 | 8 |
| Pillage Fury | 100 | 0 | 40 | 1 | 0 | 0 | 97 | 28 |
| Pillage Delay | 10 | 10 | 20 | 15 | 0 | 10 | 37 | 39 |
| Pillage Priority | 90 | 0 | 50 | 25 | 0 | 0 | 79 | 20 |
| Pillage Margin | 0 | 0 | 0 | 22 | 0 | 0 | 43 | 85 |
| Pillage Companionship | 0 | 50 | 0 | 89 | 0 | 50 | 23 | 35 |
| Pillage Friends | 50 | 0 | 0 | 23 | 0 | 0 | 85 | 41 |
| Vengence Multiplier | 100 | 100 | 100 | 42 | 0 | 100 | 51 | 38 |
| Vengence Max | 100 | 100 | 100 | 30 | 0 | 100 | 96 | 40 |
| Vengence Priority | 100 | 100 | 100 | 28 | 0 | 100 | 6 | 96 |
| Defend Friend Multiplier | 100 | 100 | 100 | 89 | 100 | 100 | 69 | 68 |
| Defend Friend Max | 100 | 100 | 60 | 91 | 100 | 100 | 83 | 32 |
| Defend Friend Priority | 80 | 90 | 75 | 10 | 100 | 90 | 5 | 40 |
| Attack Good Guys | 100 | 0 | 100 | 71 | 0 | 0 | 9 | 24 |
| Limit Give | 100 | 100 | 60 | 39 | 100 | 100 | 58 | 56 |
| Group Aware | 0 | 0 | 100 | 72 | 100 | 0 | 72 | 100 |

**Table 6: JHG assumptions and checker types.**

| Type | Justification | Assumptions | CAB-Specific? |
|---|---|---|---|
| Progress | Progress checkers are common in the AAT framework. They allow AAT to determine how well a specific generator has been performing, which has obvious benefits when deciding which generator to choose. They also provide a sense of location, in time and/or space. | Popularity increased 1 and 2 rounds after the CAB agent was last used (1 round is an immediate effect, while 2 rounds allows other players to observe the CAB's actions and form a response). | Yes. |
| | | Relative popularity increased 1 and 2 rounds after the CAB agent was last used. | Yes. |
| | | The popularity rank 1 and 2 rounds after the CAB agent was last used was "good". | Yes. |
| | | The current relative popularity is "good". | No; however, this assumption is still useful for determining our current standing in society. |
| | | The current popularity rank is "good". | No; however, this assumption is still useful for determining our current standing in society. |
| | | The current round number is 30 or smaller. | No; however, this assumption is still useful for understanding where we are situated in time. |
| | | The total number of players is 10 or smaller. | No; however, this assumption is still useful for understanding how large society is. |
| | | The CAB agent was just used. | Yes. |
| Detect Comm.'s | The first step a CAB agent performs when allocating tokens is the detection of existing communities via the Louvain algorithm. The state of the network (number and strength of positive edges, number and strength of negative edges, etc.) and the specific CAB parameters can affect how modularity is calculated and thus which communities are detected. Additionally, if there are frequent, significant changes in detected communities across rounds, it is more unlikely that the agents will select consistent, desired communities (i.e., the agents will be "wishy-washy" about which community they want to be a part of). | Considering only positive edges, the graph density is large. | No; however, the impact of this assumption will be CAB-specific because each one weights positive edges differently. |
| | | Considering only negative edges, the graph density is large. | No; however, the impact of this assumption will be CAB-specific because each one weights negative edges differently. |
| | | Modularity has not changed (increased or decreased) significantly. | Yes. |
| | | The detected communities have not changed significantly. | Yes. |
| | | The detected communities are not significantly different when using minimum magnitude of edges instead of maximum magnitude ($\hat{I}^{+}_{k,j}(\tau)$ and $\hat{I}^{-}_{k,j}(\tau)$ used in the modularity calculation). | Yes. |
| | | The detected communities are not significantly different from the detected communities of the agent that was just used. | Yes. |
| | | The detected communities are not significantly different from the detected communities the last time the agent was used. | Yes. |

| | | | |
|---|---|---|---|
| Desired Comm. | The next step CAB agents perform is the selection of a desired community. Each CAB agent weights candidate communities differently based on various metrics. Thus, the selection of a desired community is unique to each CAB agent. In this selection process, they assume the desired community is able to be formed. They also make assumptions about which metrics (modularity, familiarity, collective strength, etc.) are more valuable for selecting a community. | The collective strength of the desired community is growing (i.e., we would like our target community to be growing stronger, not weaker). | Yes. |
| | | The collective strength of the desired community is a large percentage of society's total popularity (i.e., the community has a significant presence in society). | Yes. |
| | | The collective strength of the desired community is close to the desired/target collective strength. | Yes. |
| | | The desired community is formed (contains every required player) or is at least almost formed. | Yes. |
| | | The CAB's prominence within the desired community is "good". | Yes. |
| | | The CAB's familiarity score in the desired community is better than the community's modularity (i.e., the agent has strong personal connections in the community). | Yes. |
| | | The desired community's prosocial score is "good". | Yes. |
| | | The desired community is not significantly different from the desired community of the agent that was just used. | Yes. |
| | | The desired community is not significantly different from the desired community the last time the agent was used. | Yes. |
| Keep | The third step involves determining how many tokens to keep for defense. Each CAB agent uses unique parameters to predict how other agents will attack it; accordingly, they assume their predictions are accurate. | The CAB agent does not exhaust too many of its tokens for keeping (keeping usually is not as useful as building relationships through giving or plundering through theft). | Yes. |
| | | The number of attackers is "low". | No; however, this is useful to understand the degree of enmity against us. |
| | | The cumulative popularity of the attackers is "low". | No; however, this is useful to understand how strong our defense should have been. |
| | | The defense that was used was "effective". | No; however, this is useful to understand how strong the actual defense was. |
| | | The last time the CAB agent was used and attacked, its defense was"effective". | Yes. |
| | | If the CAB agent had been used in the previous round, the defense would have been "effective". | Yes. |
| | | Of the players that attacked in the previous round, none are in the CAB agent's desired community. | Yes. |
| | | Of the players that attacked in the previous round, none are in the CAB agent's detected community. | Yes. |

| | | | |
|---|---|---|---|
| Attack | The fourth step involves determining how to attack other players. Again, each CAB agent uses unique parameters to select what type of attack to carry out, as well as which player to target and how many tokens to use. In making these decisions, the CAB agents assume the player they attack will receive damage and that they will receive a boost in popularity from the attack. | The last time the CAB agent was used, its attacks damaged the targeted player. | Yes. |
| | | The last time the CAB agent was used, its attacks helped increase its popularity. | Yes. |
| | | The last time the CAB agent was used, its popularity did not decrease 2 rounds after an attack (i.e. the attack was justified and/or did not cause significant retaliation). | Yes. |
| | | The CAB agent estimates that, if it had been used in the previous round, its attacks would have damaged the targeted player. | Yes. |
| | | The CAB agents estimates that, if it had been used in the previous round, its attacks would have helped increase its popularity. | Yes. |
| | | The CAB agent does not exhaust too many of its tokens for attacking (building relationships through giving can be valuable, and it might be useful to keep some tokens for defense). | Yes. |
| | | Of the players attacked in the previous round, none are in the CAB agent's desired community. | Yes. |
| | | Of the players attacked in the previous round, none are in the CAB agent's detected community. | Yes. |
| | | We received popularity from any attacks in the previous round. | No; however, this is useful to understand the effectiveness of current attacks. |
| | | We damaged the other player, if any, in the previous round. | No; however, this is useful to understand the effectiveness of current attacks. |
| Give | In the final step, each CAB agent determines how many tokens to give to players in its desired community. Again, specific parameters are used, so token allocations are unique to each CAB agent. In choosing who to give to and how much to give, CAB agents assume that these players will reciprocate at least an equal (or close to equal) amount of positive influence. They also assume that those in the desired community they do not give to will not (or cannot) reciprocate. | The CAB agent does not allocate all of its tokens for giving (while giving is arguably the best action, it can still be useful to keep and/or attack). | Yes. |
| | | The CAB agent gives to multiple players (i.e., it tries to build various relationships). | Yes. |
| | | The CAB agent gives tokens to all (or most) of the players in its desired community (it is probably more difficult to form/maintain the community if tokens are only given to some members). | Yes. |
| | | Of the players the CAB agent gives tokens to, all (or most) reciprocate an equal or larger amount of popularity. | Yes. |
| | | Of the players the CAB agent gives tokens to, none (or relatively few) have attacked other players (i.e., attacking creates enmity, so it can be bad to be associated with a disliked player). | Yes. |
| | | Of the players the CAB agent gives tokens to, none (or relatively few) have attacked us (i.e. if a player has attacked us, they might be prone to do it again in the future). | Yes. |
| | | Of the players that tokens were given to in the previous round, all (or most) are in the CAB agent's desired community. | Yes. |
| | | The last time the CAB agent was used, all (or most) of the players it gave to reciprocated an equal or larger amount of popularity within 2 rounds. | Yes. |

## 6.3 Raw Features

From [9], define $P(\tau)$ as a vector of $n$ popularity scores in round $\tau$, where $n$ is the number of players in the game. Define $I(\tau)$ as the $n \times n$ influence matrix (i.e., how each player has impacted the popularities of all other players) in round $\tau$. Define $x_i(\tau)$ as a vector of $n$ token values, where each entry in the vector represents player $j$'s proportion of tokens used to give to or steal from player $i$; stolen tokens are negative values.

With the previously-defined values, we can now define the raw features we used in the JHG. Let $I_f(\tau)$ denote the flattened, one-dimensional representation of $I(\tau)$. Our raw feature vectors are defined as $I_f(\tau) \parallel P(\tau) \parallel x_i(\tau)$, where the operator "$\parallel$" denotes concatenation (e.g., $\langle x_1, x_2 \rangle \parallel \langle y_1, y_2 \rangle = \langle x_1, x_2, y_1, y_2 \rangle$). An important item to note is that the dimensions of $I_f(\tau)$, $P(\tau)$, and $x_i(\tau)$ all depend on the number of players $n$ in the game/society. Because game sizes can vary and we experimented with different sizes, padding is necessary in order to ensure consistent raw feature vector dimensions. We accounted for a maximum number of players of 30 and used zero padding.

## 6.4 Training

In our training, we occasionally would insert coordinated assassin agents. Specifically, these agents are defined in [9] and are called Conspiring Autonomous Thieves (CATs). A CAT agent is designed to work together with other CATs in order to pillage non-CAT players. CATs attempt to identify each other by keeping all of their tokens in the first round. In subsequent rounds, CATs will attack the weakest (lowest popularity) player that did not keep tokens in the first round. CATs will keep tokens if no targets are available/identifiable or if they estimate that only a portion of their tokens is required for an attack. With probabilities 0.9, 0.05, and 0.05 we randomly inserted 0, 1, and 2 CAT agents, respectively, into the society. We also used seeding to ensure that the training and testing conditions for each algorithm were identical.

In another part of our training process, we paired agents with randomly-selected CABs that were tuned in specific scenarios. The tuning process involves a genetic algorithm that is identical to that found in [9]. We tuned CABs with 0, 1, and 2 CAT agents and ran 200 iterations of the genetic algorithm. At the end of each iteration, the algorithm stores the parameters of numerous CAB agents, sorted by performance in descending order. For our selection, we randomly picked an iteration from the final 50 iterations (i.e., iterations 151 to 200) and chose the top/best CABs from that iteration.

## 6.5 Adaptability Score Components

Here we define the specific values of $U_d^*$, $U_{\min}$, $U_c^*$, and $v_{\text{mm}}$ we used in the JHG.

In the JHG, we used two different competitive scenarios: (1) societies composed of players that always keep the entirety of their tokens and (2) societies composed of players that steal an equal amount of tokens from every other player. The $U_d^*$ values for these two conditions, respectively, were 73.970 and 171.478 (rounded to three digits). The $U_{\min}$ value was 0 in both conditions, as agents cannot receive a negative popularity score in the JHG. To test for cooperation, we used three cooperative scenarios: (1) self-play, (2) societies where half the players were copies of the agent being tested and the other half were tit-for-tat agents, and (3) societies where every player but the agent being tested was a tit-for-tat agent. The $U_c^*$ values for these three conditions, respectively, were 397.487, 400.353, and 383.146 (rounded to three digits). The $v_{\text{mm}}$ value was 0 in every condition (since agents can only guarantee themselves a popularity score of 0).

The $U_{\min}$ and $v_{\text{mm}}$ values were found through through the theoretical components of the game. The $U_d^*$ and $U_c^*$ values were found through empirical simulation.

## 6.6 Additional Results

In the main paper, for the sake of space we only included certain results that we felt were the most illuminating. Here we present the competitive, cooperative, and adaptability scores for each algorithm in the JHG. Figures 13-15 show the algorithms' average scores (black bars show standard errors).

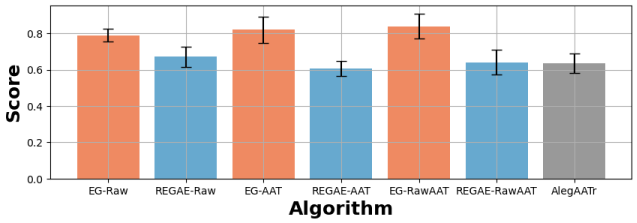



**Figure 13: Competitive scores.**

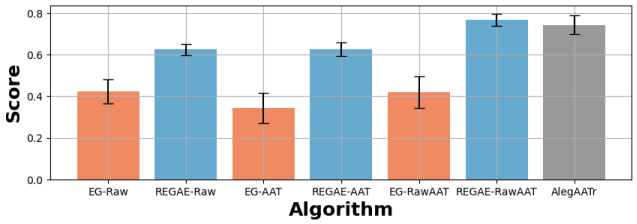



**Figure 14: Cooperative scores.**

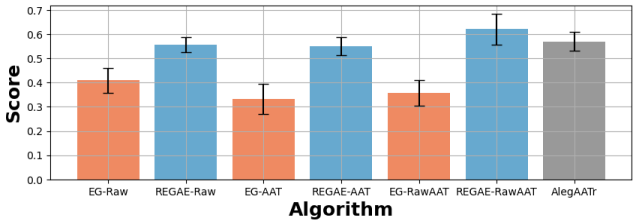



**Figure 15: Adaptability scores.**

## REFERENCES


[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. https://www.tensorflow.org/ Software available from tensorflow.org.

[2] S. Barrett, P. Stone, and S. Kraus. 2011. Empirical evaluation of ad hoc teamwork in the pursuit domain. In *The 10th International Conference on Autonomous Agents and Multiagent Systems-Volume 2*. 567–574.

[3] Maxim Egorov. 2016. Multi-agent deep reinforcement learning. *CS231n: convolutional neural networks for visual recognition* (2016), 1–8.

[4] A. Gautam, T. Whiting, X. Cao, M. A. Goodrich, and J. W. Crandall. 2022. A Method for Designing Autonomous Robots that Know Their Limits. In *International Conference on Robotics and Automation (ICRA)*. 121–127.

[5] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, et al. 2013. *An introduction to statistical learning*. Vol. 112. Springer.

[6] Ethan Pedersen and Jacob W. Crandall. 2023. AlegAATr the Bandit. In *Proceedings of the* 26th *European Conference on Artificial Intelligence*. 1867–1874.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[9] J. Skaggs, M. Richards, M. Morris, M. A. Goodrich, and J. W. Crandall. 2024. Fostering Collective Action in Complex Societies using Community-Based Agents. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. Jeju, South Korea.

[10] Peter D Taylor and Leo B Jonker. 1978. Evolutionary stable strategies and game dynamics. *Mathematical biosciences* 40, 1-2 (1978), 145–156.